Spiking, Recurrent, and Attention Autoencoders for Temporal Signals: A Leakage-Controlled Nested Cross-Validation Comparison

André Furlan
Instituto de Biociências, Letras e Ciências Exatas - UNESP
andre.furlan@unesp.br

***Abstract*—Spiking Neural Networks (SNNs) promise low-power temporal signal processing through event-driven, sparse computation, but their use as autoencoders for waveform reconstruction lacks a controlled, leakage-free empirical comparison against modern non-spiking baselines. We compare four trained autoencoder families — a spiking autoencoder (SNN-AE), an LSTM autoencoder (LSTM-AE), a GRU autoencoder (GRU-AE), and a bottlenecked Transformer autoencoder (Transformer-AE) without encoder–decoder cross-attention — under three spike-encoding front ends (direct, Poisson, latency), together with PCA and mean-frame linear reference points. All models share a single C++/CPU harness [1] for matched timing and a transparent operation-count cost budget. Evaluation uses *nested six-fold leave-one-speaker-out cross-validation* on the Free Spoken Digit Dataset (FSDD) [2]: no source recording or speaker crosses a split, and SNN hyperparameters are selected inside each fold on the held-out validation speaker only, then retrained on the combined training and validation data before a single evaluation on the untouched test speaker. Model differences are assessed with a recording-level paired analysis (bootstrap confidence intervals over recordings, Wilcoxon signed-rank, Holm–Bonferroni), with speaker-level ($n = 6$) and seed-level results reported as generalization- and optimization-robustness evidence. We report reconstruction quality (MSE, MAE, R^2) as mean $\pm$ standard deviation over seeds, a raw per-class operation count as the primary cost figure (with a weighted proxy as a labelled sensitivity analysis), the self-attention $O(T^2 d_{\text{model}})$ interaction cost, and real parameter counts. The harness and profiles are released for reproduction.**

***Index Terms*—spiking neural networks, LSTM, GRU, Transformer, autoencoder, temporal signals, surrogate gradients, nested cross-validation, data leakage, audio signal processing**

I. Introduction

One-dimensional temporal signals such as audio waveforms, biosignals, and sensor streams often need to be processed and interpreted in low-power embedded systems where the energy budget is a primary constraint.

Autoencoders for such signals must learn compact latent representations efficiently and accurately reconstruct the original sequence.

Long Short-Term Memory (LSTM) autoencoders are a well-established baseline for sequence reconstruction [3], [4], but their dense matrix operations are computationally expensive. Spiking Neural Networks (SNNs) otherwise communicate via sparse binary events and perform synaptic updates only when spikes occur, offering potential energy reduction on general-purpose and neuromorphic hardware [5]–[7].

Training SNNs for reconstruction requires backpropagation through non-differentiable spike functions which behave like step functions. Surrogate gradient methods [8], [9] enable end-to-end gradient-based training and have produced competitive SNN classification performance [6], [10]. However, a controlled study of SNN autoencoders for temporal signal reconstruction — compared against several modern non-spiking baselines, under an evaluation protocol that provably prevents information from the test signals leaking into training or hyperparameter selection — is lacking.

Evaluation rigor is the crux. When fixed-length windows from all recordings are pooled, shuffled, and split, windows from the same utterance — and the same speaker — appear in both training and validation, so reported errors partly measure memorization of speaker- and recording-specific waveform detail. Speaker identity is the natural unit of generalization for a spoken-digit benchmark, and FSDD provides only six speakers, which also bounds how strong any population-level claim can be.

This paper addresses these gaps with four contributions:

1) A leakage-controlled protocol: *nested six-fold leave-one-speaker-out cross-validation* on FSDD [2] in which no source recording or speaker crosses a split and SNN hyperparameters are selected on the inner validation speaker only.
2) A comparison of four trained autoencoder families under three input encodings — SNN-AE, LSTM-AE, GRU-AE, and a bottlenecked Transformer-AE without encoder–decoder cross-attention — plus PCA and mean-frame linear reference points, all on one matched C++/CPU harness [1], [11].
3) A hierarchical statistical analysis with the recording-level paired difference as the primary estimand (bootstrap over recordings, Wilcoxon, Holm–Bonferroni), and speaker-level ($n = 6$) and seed-level results reported as robustness rather than population inference.
4) An analytic computational-cost budget reported as a raw per-class operation count (primary) with a weighted proxy as a labelled sensitivity analysis, alongside the self-attention $O(T^2 d_{\text{model}})$ term and real parameter counts; the reproducible framework is released.

II. Related Work

A. Recurrent and attention sequence autoencoders

LSTM autoencoders encode variable-length sequences into fixed-size latent vectors and reconstruct the original sequence from the latent code [3], [4]. The constant-error carousel in the cell recurrence mitigates vanishing gradients during backpropagation through time (BPTT) [12], [13]. The Gated Recurrent Unit (GRU) [14] merges the input and forget gates and drops the separate cell state, giving a smaller recurrent cell with performance comparable to the LSTM on many sequence tasks [15]. Self-attention encoders [16] replace recurrence with a content-based $O(T^2)$ mixing operation and layer normalization [17]. We use all three as trained non-spiking baselines; the Transformer variant is *bottlenecked* — the decoder is conditioned only on the fixed-dimension latent, with no encoder–decoder cross-attention — so that, like PCA and the recurrent autoencoders, all reconstruction information must pass through the latent.

B. Spiking neural networks and surrogate gradients

SNNs model computation via binary spike events governed by membrane potential dynamics [5], [7]. The Leaky Integrate-and-Fire (LIF) neuron is the most widely deployed model. Training with backpropagation requires differentiable surrogates for the discontinuous Heaviside activation; surrogate gradient methods [8], [18], [19] have enabled competitive SNN accuracy with lower inference cost. SNN research has also expanded from classification to temporal sequence modelling and representation learning, but systematic reconstruction benchmarks remain limited [10], [19].

C. Spike encoding for temporal signals

Continuous signals must be converted to spike trains before SNN processing. Rate coding (Poisson spike generation proportional to amplitude), latency coding (earlier spikes for stronger stimuli), and direct pass-through of normalized values represent the three dominant strategies [8], [20]. Encoding choice affects sparsity, gradient flow, training stability, and reconstruction fidelity [8].

D. Evaluation protocol and leakage

On speaker-labelled audio benchmarks, pooling windows before splitting places windows from the same utterance and speaker on both sides of the split, so measured error conflates generalization with memorization. Speaker-disjoint (leave-one-speaker-out) evaluation is the standard remedy; when model or hyperparameter selection is also involved, a *nested* scheme is required so that the test speaker never influences selection or early stopping [21]. With only six FSDD speakers, we treat the speaker-level result as a robustness check and make the recording-level paired difference the primary estimand.

III. Methodology

A. Problem formulation

Let $x \in \mathbb{R}^T$ denote a temporal signal window of length T. An autoencoder learns encoder $E : \mathbb{R}^T \to \mathbb{R}^d$ and decoder $D : \mathbb{R}^d \to \mathbb{R}^T$ minimizing reconstruction loss:

$$\mathcal{L} = \|x - D(E(x))\|_2^2. \tag{1}$$

Where x is the input window, $E(x)$ is the latent representation, and $D(E(x))$ is the reconstructed output.

All trained autoencoders use latent dimension $d = 32$; the recurrent models (SNN-AE, LSTM-AE, GRU-AE) use hidden size $H = 64$ and the Transformer-AE uses $d_{\text{model}} = 64$ (Sec. III-D). The window is reshaped into T/f frames of f samples before entering the sequence models. PCA and a mean-frame predictor provide linear reference points, fitted per fold on training data only.

B. LIF neuron model

The continuous-time Leaky Integrate-and-Fire neuron is described by [7]:

$$\tau \frac{dV}{dt} = -(V - V_{\text{rest}}) + R\,I(t), \qquad \tau = RC. \tag{2}$$

Where V is the membrane potential, V_{rest} is the resting potential, $I(t)$ is the input current, R is the membrane resistance, and C is the membrane capacitance. The time constant τ governs the rate of potential decay. The $R\,I(t)$ term is consistent with the discrete update in Eq. (3).

Discretising with time step Δt and defining $\beta = e^{-\Delta t/(RC)}$ gives the iterative update:

$$V[t] = \beta\, V[t-1] + R\, I[t]. \tag{3}$$

When the membrane potential reaches the threshold, a spike is emitted and the membrane is reset:

$$s[t] = \mathbf{1}[V[t] \geq V_{th}], \qquad V \leftarrow \begin{cases} 0 & \text{(hard reset)} \\ V - V_{th} & \text{(soft reset)} \end{cases}. \tag{4}$$

Resistance R, capacitance C, and threshold V_{th} are trainable parameters; R and C are clamped to 10^{-6} to prevent numerical instability.

Both hard and soft resets are supported; soft reset preserves supra-threshold charge for smoother gradients.

C. Surrogate gradient training

The spike function $s = \mathbf{1}[V \geq V_{th}]$ has true gradient zero almost everywhere, blocking standard backpropagation [8].

Surrogate gradient methods replace $\partial s/\partial V$ with a smooth approximation in the backward pass while preserving the exact Heaviside rule in the forward pass [8], [18].

Exponential (SuperSpike [9]):

$$\hat{\sigma}'(V) = \frac{1}{\sigma_s} \exp\left(-\frac{|V - V_{th}|}{\sigma_s}\right). \tag{5}$$

Exponential surrogate gradient. σ_s controls the width of the approximation; $\sigma_s = 1$ is used in this work.

Boxcar:

$$\hat{\sigma}'(V) = \mathbf{1}\big[|V - V_{th}| < \tfrac{w}{2}\big]. \tag{6}$$

Boxcar surrogate gradient. w controls the width of the approximation.

We use the exponential surrogate ($\sigma_s = 1$). BPTT [12] unrolls both LSTM and SNN through the full time dimension T; the reverse-time recurrence for layer l is:

$$\delta_l[t] = \hat{\sigma}'(V[t]) \odot \big(W_{l+1}^{\top}\, \delta_{l+1}[t]\big) + \beta\, \delta_l[t+1], \tag{7}$$

where $\delta_l[t] \in \mathbb{R}^{n_l}$ is the error signal at layer l and time t, $W_{l+1} \in \mathbb{R}^{n_{l+1} \times n_l}$ is the feed-forward weight matrix mapping layer-l activations to layer-$(l+1)$ pre-activations (so the forward pass computes $W_{l+1}\, a_l$), $\odot$ is the element-wise product, $\hat{\sigma}'(V[t])$ is the surrogate derivative of Sec. III-C, and $\beta = \partial V_{\text{post}}/\partial V_{\text{pre}} = e^{-\Delta t/(RC)}$ carries the error one step back in time through the membrane recurrence. The spatial term $W_{l+1}^{\top}\delta_{l+1}[t]$ and the temporal term $\beta\, \delta_l[t+1]$ are the SNN analogue of the standard BPTT split.

D. Recurrent and attention baselines

LSTM-AE. Standard gate equations — input, forget, and output gates from a dense pre-activation, with cell-state accumulation $C_t = f_t \odot C_{t-1} + i_t \odot g_t$ [4], [13]. The encoder stacks two LSTM layers ($H = 64$); the final hidden state is projected to the $d = 32$ latent by $\text{Linear}(H \to d) \to \tanh$. The decoder applies $\text{Linear}(d \to H)$, replicates the result over time, runs two stacked LSTM layers, and projects each step back to the frame. States reset between samples.

GRU-AE. Identical topology with the LSTM cell replaced by a GRU cell [14], [15]: reset gate r_t, update gate z_t, and candidate $n_t = \tanh\big(x_t W_n^{\top} + r_t \odot (h_{t-1} U_n^{\top}) + b_n\big)$, $h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$. Dropping the cell state removes one gate's worth of parameters per layer; the real counts are reported in Table I.

Transformer-AE (bottlenecked). The encoder embeds each frame to $d_{\text{model}} = 64$, adds sinusoidal positional encodings, applies N post-norm self-attention blocks [16], [17], mean-pools over time, and projects to the latent through a $\tanh$. The decoder expands the latent, replicates it over T/f positions, adds positional encodings, applies N self-attention blocks, and projects back to frames. There is *no encoder–decoder cross-attention*: this is a deliberate design choice so that reconstruction must pass through the fixed-dimension latent, making the model a strict-compression autoencoder directly comparable to PCA and the recurrent autoencoders. Its parameter count is chosen *near* the recurrent baselines, not matched; the actual value is reported in the results.

Sequence-length cost. Per input window, the recurrent baselines cost $O(TdH)$ operations while self-attention costs $O(T^2 d_{\text{model}})$ for the score and context products plus $O(T d_{\text{model}}^2)$ for the projections. At $T/f = 32$ frames this quadratic interaction term is modest, but it is an architecture-level cost that is independent of the reconstruction-error ranking and is reported explicitly in the operation-count budget (Sec. III-I).

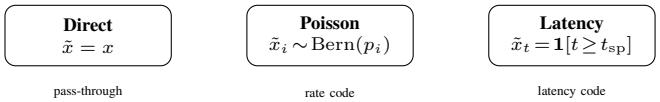

Figure 1. Three spike encoding schemes. Direct passes raw values; Poisson generates stochastic binary spikes proportional to amplitude; latency fires at a time inversely proportional to stimulus strength.

E. Spike encodings

Three strategies convert raw window x into pre-processed $\tilde{x}$:

Direct. $\tilde{x} = x$. Normalized values are passed without binarization.

Poisson. Each element is independently binarized via Bernoulli sampling [8]:

$$\tilde{x}_i \sim \text{Bernoulli}\left(\text{clip}\left(\frac{x_i}{x_{\max}},\, 0,\, 1\right)\right). \tag{8}$$

Where $x_{\max}$ is the maximum value in the window after z-score normalization, ensuring the Bernoulli parameter is in $[0, 1]$.

Latency. A binary step fires once per element from a time inversely proportional to signal strength [20]:

$$\tilde{x}_{t,i} = \mathbf{1}\left[t \geq \left\lfloor \left(1 - \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}\right)(T-1) \right\rfloor\right]. \tag{9}$$

Where $x_{\min}$ and $x_{\max}$ are the minimum and maximum values in the window after z-score normalization, respectively. Stronger signals fire earlier; weaker signals fire later.

F. SNN pre-processing modes

After spike encoding, a pre-processing transform ϕ is applied before the autoencoder input.

Dense. $\phi(\tilde{x}) = \tilde{x}$. No additional transform.

Conv1d. A fixed 3-tap temporal smoothing filter:

$$\phi(\tilde{x})_t = 0.25\, \tilde{x}_{t-1} + 0.5\, \tilde{x}_t + 0.25\, \tilde{x}_{t+1}, \tag{10}$$

with boundary clamping. Reduces high-frequency spike jitter before autoencoder input.

Recurrent. A recurrent LIF cell integrates the encoded signal:

$$v[t] = \alpha\, v[t-1] + \tilde{x}[t] - s[t-1]\, V_{th}, \tag{11}$$

$$s[t] = \mathbf{1}[v[t] \geq V_{th}], \tag{12}$$

where α and V_{th} are sweep parameters.

G. SNN autoencoder architecture

The SNN autoencoder (Fig. 2) is a two-layer fully connected network:

$$\text{Enc:}\quad \text{Lin}(T \to H) \to \text{Leaky} \to \text{Lin}(H \to d) \to \text{Leaky}$$

$$\text{Dec:}\quad \text{Lin}(d \to H) \to \text{LeakyInt} \to \text{Lin}(H \to T)$$

with $T = 256$, $H = 64$, $d = 32$.

Leaky layers emit binary spikes; LeakyInt operates in readout mode, emitting membrane potential v directly to enable

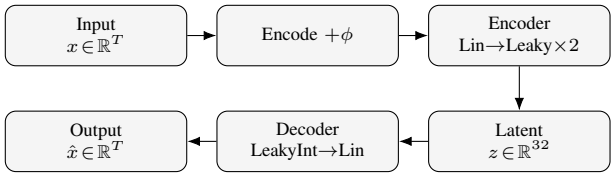

Figure 2. SNN autoencoder architecture. Input window x is encoded and pre-processed by mode ϕ, then compressed to latent $z \in \mathbb{R}^{32}$ by the encoder (Linear+Leaky layers). The decoder (LeakyInt+Linear) reconstructs $\hat{x}$.

continuous-valued reconstruction. The discrete LIF update follows Eq. (3) with $R = 1/V_{th}$ and $C = -1/\ln(\alpha)$ derived from sweep parameters. All membrane states are reset between independent samples.

The four trained families are: SNN-AE (the spiking autoencoder above), LSTM-AE, GRU-AE, and Transformer-AE (Sec. III-D). The spiking pre-processing choices — dense (ϕ identity), conv1d (fixed 3-tap smoothing), recurrent (the LIF input transform above) — are *input transforms* to the one SNN autoencoder, not separate families: the encoder/decoder stack and latent size are fixed and only ϕ changes. Which transform is used for a given fold is chosen by the per-fold selection procedure (Sec. III-D), so SNN-AE denotes the selected variant.

H. Training

All trained models minimize MSE reconstruction loss via Adam [22] ($\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$). SNN biophysical parameters (R, C, V_{th}) use a scaled learning rate $\eta_{\text{bio}} = 0.1\,\eta$ to stabilize membrane-dynamics optimization [8]; R and C are clamped to 10^{-6} in the forward pass with gradients zeroed in the clamped region. BPTT [12] propagates gradients through the full time dimension. Early stopping monitors a held-out monitor MSE with patience $P = 5$ epochs and minimum delta 10^{-8} over a maximum of 30 epochs [10]. Batch size is 1 to accommodate stateful sequence processing. Each configuration is trained with five random seeds (42–46).

Per-fold hyperparameter selection (SNN). Within each outer fold, the full SNN grid — pre-processing mode $\in$ {dense, conv1d, recurrent}, threshold $V_{th} \in \{0.5, 1.0, 1.5\}$, membrane decay $\alpha \in \{0.8, 0.9, 0.99\}$, three encodings — is trained on that fold's four training speakers and scored on the inner validation speaker. The configuration with the lowest validation MSE is selected, then *retrained* on the training and validation speakers combined and evaluated *once* on the held-out test speaker. Because train and validation speakers share no recordings with the test speaker but not with each other, the retrain's early-stopping monitor is a recording-disjoint slice carved (seeded, whole recordings) from the training speakers, so no test-speaker signal ever reaches a fitting or stopping decision. The per-fold selected configuration and its inner-validation scores are written to a selection manifest and summarized in Table III.

Fixed-architecture baselines. LSTM-AE, GRU-AE, and Transformer-AE use fixed profile architectures; their only use of the validation speaker is early stopping, which never touches the test speaker. PCA and the mean-frame predictor are fitted per fold on the training speakers only.

I. Analytic computational cost

We report an analytic *computational-cost* figure, not an energy measurement. The primary quantity is the raw operation count

$$C_{\text{raw}} = \sum_k N_k, \tag{13}$$

where N_k is the number of operations of class k executed in one forward pass over a window. We count, separately, the following classes: for the ANN models, dense multiply–accumulates (MACs), nonlinearity evaluations, attention $QK^\top/\cdot V$ products, and normalization operations; for the SNN, synaptic accumulations, neuron-state updates, and spike/event operations. The event-driven classes are scaled by the measured mean spike fraction $r \in [0, 1]$ of the evaluated run, so an SNN that never spikes contributes no event-operation cost.

As a secondary, explicitly labelled *sensitivity analysis* we also report a weighted proxy

$$C_{\text{weighted}} = \sum_k w_k N_k, \tag{14}$$

under a stated choice of relative per-class weights w_k; the ranking of models under C_{weighted} is reported only alongside the weights that produce it, never as a standalone claim.

Neither C_{raw} nor C_{weighted} is energy. Real energy on a given device is dominated by memory movement, cache and SIMD behaviour, clock frequency, supply voltage, exploited sparsity, and the target silicon (CPU, GPU, or neuromorphic). On-device power measurement is future work; the figures here are design-time guidance for comparing architectures under matched software and a single CPU backend.

IV. Experimental Setup

Dataset.

The Free Spoken Digit Dataset (FSDD) [2] contains recordings of spoken digits (0–9) from six speakers at 8 kHz. Each recording is partitioned into non-overlapping windows of $T = 256$ samples (32 ms) with stride $S = T$, so consecutive windows share no samples; the full dataset is used (no window-count cap). Adjacent windows of one recording remain temporally correlated, which motivates the recording-level statistical unit (Sec. IV-A).

Preprocessing.

Each window is z-score normalised in place (subtract mean, divide by standard deviation) before encoding — a per-window statistic, so it introduces no cross-window leakage. Encodings that require a $[0, 1]$ range (Poisson, latency) re-normalise the z-scored window to its own $(x_{\min}, x_{\max})$ range internally. The three encodings are deterministic transforms (Poisson uses a per-window seeded draw) and are not fitted.

Nested leave-one-speaker-out cross-validation.

The six speakers are assigned deterministic indices 0–5. For outer fold $f \in \{0, \ldots, 5\}$ the test speaker is f, the validation speaker is $(f + 1) \bmod 6$, and the remaining four

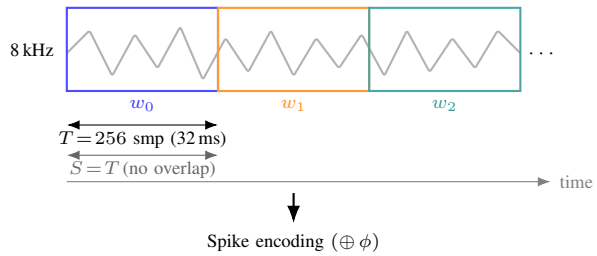

Figure 3. Signal windowing. Each recording is segmented into non-overlapping windows of $T = 256$ samples (32 ms at 8 kHz). Stride $S = T$ yields windows with no shared samples. Windows are assigned to train / validation / test splits by speaker before any pooling.

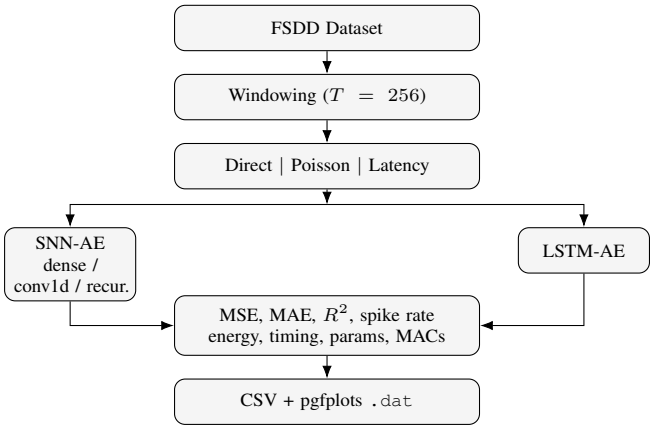

Figure 4. Experiment execution pipeline. Per outer fold, each FSDD window is encoded three ways and processed by the four trained families (SNN-AE with a per-fold selection sweep; LSTM-AE, GRU-AE, Transformer-AE fixed) plus the PCA and mean-frame references. Per-window errors and manifests are written to CSV and JSON.

speakers are the training set. Every window is assigned to a split *by its speaker, before any pooling*; training windows are shuffled (seeded), validation and test windows kept in (recording, position) order. Each speaker is the test speaker exactly once and the validation speaker exactly once. At run start the harness asserts that no `recording_id` and no speaker appears in more than one split and writes a per-fold split manifest (speaker sets, per-recording window counts, content hashes); a violation aborts the run. Headline numbers come only from the test-speaker split, aggregated across the six folds.

Evaluation harness.

The C++ harness [1] runs on the XTensor backend [11] (CPU-only, CBLAS, SIMD via xsimd). All models — spiking and non-spiking — are trained and timed on this one backend, so the training and inference times in Table I are directly comparable.

Metrics.

Per run we report reconstruction quality (MSE, MAE, R^2) and, for cost, spike rate (SNN only; 0 for the non-spiking models), the raw per-class operation count of Sec. III-I, parameter count, training time, and inference time in ms. FSDD provides no anomaly labels, so classification metrics are out of scope. Reconstruction error in the headline tables is the mean $\pm$ sample standard deviation over the five seeds (per-seed means first); the inferential analysis is in Sec. IV-A. A per-window error record (model, encoding, fold, split, speaker, recording, window, MSE, MAE) is written for every model so that paired comparisons can be formed at the window, recording, speaker, or seed level.

Profile.

A single JSON profile (`article-loso.json`) configures the harness: full FSDD, six outer folds, five seeds, the four trained families plus the two linear references, three encodings, and the SNN selection grid ($3 \times 3 \times 3$). The run script iterates the outer fold index; a profile audit test verifies the profile parses and validates.

A. Statistical methods

Comparisons are paired on a shared, model-independent window identifier, so every model reconstructs the same windows and differences are formed per window. Let $\mathrm{MSE}_{m,i}$ be the error of model m on window i.

Primary — recording level. For a recording r, let $E_{m,r} = \mathrm{mean}_{i \in r}\, \mathrm{MSE}_{m,i}$. For model m and reference b the paired difference is $d_r = E_{m,r} - E_{b,r}$. We report $\mathrm{mean}(d_r)$, a 95% confidence interval from a bootstrap that resamples recordings, the Wilcoxon signed-rank p-value on $\{d_r\}$, and Holm–Bonferroni correction across the reference set $b \in \{\text{LSTM-AE, GRU-AE, Transformer-AE, PCA, Mean}\}$. Effect size is reported both absolute ($\mathrm{mean}(d_r)$) and relative ($\mathrm{mean}(d_r)/\,\mathrm{mean}(E_{b,\cdot})$). Recordings are the smallest unit across which windows are not, by construction, near-duplicates.

Robustness — speaker level. Per held-out speaker (one per fold, $n = 6$) we form the mean error and run a paired Wilcoxon / sign test. This checks whether the sign and rough magnitude of a difference hold across the six FSDD speakers; with $n = 6$ it is not strong population-level inference and is reported as such.

Descriptive — window level. Paired bootstrap on all windows, reported but explicitly labelled pseudoreplicated (adjacent windows of a recording are correlated), so it is never the basis of a claim.

Robustness — seed level. Paired Wilcoxon across the five per-seed mean errors, characterizing optimization/reproducibility variation around the estimate.

The per-run CSVs are immutable; aggregation and statistics are separate downstream steps that never overwrite them. Synthetic known-answer tests (identical predictions $\Rightarrow p \approx 1$ and CI spanning 0; injected constant offset $\Rightarrow$ significant with CI excluding 0 and recovered offset; label permutation $\Rightarrow$ approximately uniform p) guard the analysis code in continuous integration.

V. Results

All numbers in this section come from the held-out test speaker only, pooled across the six outer folds, and are reported

as mean ± sample standard deviation over the five seeds. Reconstruction quality is in Table I (per model) and Table II (per model × encoding); Fig. 5 plots the per-encoding test MSE with seed error bars. Table III records which SNN pre-processing mode won on each fold's validation speaker, and Table IV gives the recording-level paired analysis.

A. Reconstruction quality

Table I reports test-speaker MSE, MAE, and R^2 for the four trained families and the two linear references, with each model's real parameter count and raw operation count C_{raw} (Sec. III-I). The mean-frame predictor and PCA fix the scale of a "no-model" and a "best linear" reference; a trained model is only interesting to the extent it beats PCA. Table II breaks the same quantities down by input encoding.

Interpretation is deferred to the regenerated tables: the earlier, leakage-affected version of this study over-stated the SNN advantage because train and validation windows shared speakers and recordings with the test windows. Under speaker-disjoint nested evaluation the reference points (PCA, mean-frame) and the recording-level intervals in Table IV determine which differences survive.

B. Per-fold model selection

Table III lists, for each outer fold and encoding, the SNN pre-processing mode, threshold, and decay selected on that fold's validation speaker. Because selection is redone inside every fold, the SNN result is the performance of a *selection procedure*, not of a single hand-picked configuration; disagreement of the selected mode across folds is itself a finding about how encoding-dependent the best pre-processing is.

C. Computational cost and timing

Table I also carries wall-clock training and inference time, measured for every model on the one XTensor/CPU backend, and the raw operation count. The recurrent baselines and the SNN cost $O(TdH)$ per window; the Transformer additionally pays the $O(T^2 d_{\text{model}})$ self-attention interaction term, which is reflected in its C_{raw} column independently of its MSE. The weighted proxy C_{weighted} (Eq. (14)) is reported in the released JSON as a labelled sensitivity analysis and is not used for any claim here.

VI. Discussion

Two things should be read from Table IV rather than from raw MSE gaps. First, whether each trained family beats the PCA reference at all: under speaker-disjoint evaluation, a family whose recording-level interval against PCA includes zero has not demonstrated that its nonlinearity buys anything on this benchmark. Second, the ordering among SNN-AE, LSTM-AE, GRU-AE, and Transformer-AE — a difference is only asserted where the Holm-corrected interval excludes zero. The seed-level and speaker-level ($n = 6$) checks in the released JSON say whether an ordering that holds in the pooled recording-level analysis also holds run-to-run and speaker-to-speaker.

The encoding comparison (Table II) is where an SNN-specific effect is most likely to appear, because the encoding only reshapes the SNN's input; the non-spiking baselines consume the same framed window regardless. If latency or Poisson coding helps the SNN relative to direct coding, that is a property of spike-based representation, not of the reconstruction task.

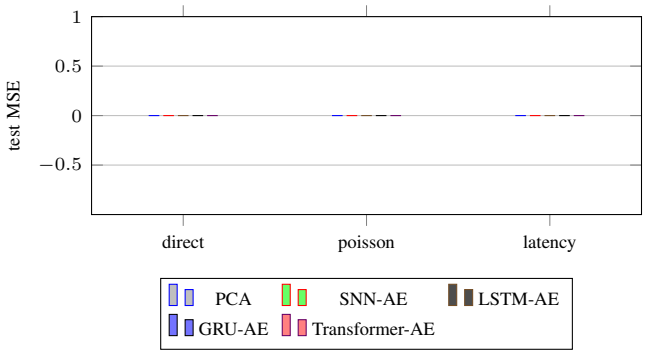

Figure 5. Test-speaker MSE by model and input encoding (mean over seeds and folds; the mean-frame reference is omitted for scale). Error bars, where shown, are ±1 seed standard deviation.

The cost picture is separable from the quality picture. The operation-count column of Table I orders the models by arithmetic work under matched software; the self-attention $O(T^2 d_{\text{model}})$ term makes the Transformer's per-window cost grow faster in sequence length than the recurrent models', independent of which model reconstructs best. Wall-clock timing on the shared CPU backend is reported alongside, but it reflects this implementation and backend, not silicon-level energy.

Limitations.

- **Six speakers.** FSDD has six speakers, so the speaker-level analysis has $n = 6$ and is a robustness check, not population inference; the recording-level analysis is the primary estimand for this reason.
- **Single dataset.** Only FSDD is used; the ranking may not transfer to other 1-D signal domains.
- **Cost is not energy.** C_{raw} and C_{weighted} count operations under matched software; real energy depends on memory movement, sparsity exploitation, and target hardware. On-device measurement is future work.
- **Correlated windows.** Non-overlapping windows of one recording remain correlated, so window-level statistics are pseudoreplicated and used descriptively only.
- **Approximate parameter matching.** The Transformer-AE is sized *near*, not equal to, the recurrent baselines; Table I reports the real counts so cost and quality can be read together.

All results are loaded directly from experiment-generated CSV files; the per-run CSVs are immutable and aggregation never overwrites them.

VII. Conclusion

We compared four trained autoencoder families — SNN-AE, LSTM-AE, GRU-AE, and a bottlenecked Transformer-AE — under three spike-encoding front ends, against PCA

Table I

TEST-SPEAKER RECONSTRUCTION AND COST BY MODEL, POOLED OVER THE SIX OUTER FOLDS (MEAN ± STD OVER FIVE SEEDS; BEST PER COLUMN IN BOLD). P IS THE REAL TRAINABLE-PARAMETER COUNT; C_{raw} IS THE RAW PER-FORWARD-PASS OPERATION COUNT OF SEC. III-I. PCA AND MEAN ARE ANALYTIC REFERENCES (NO TRAINING, NO PARAMETER COUNT).

Model	MSE	MAE	R^2	P	C_{raw}	Train (ms)	Infer (ms)
Mean	0.0000±0.0000	0.0000±0.0000					
PCA	0.0000±0.0000	0.0000±0.0000					
SNN-AE	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000	0	0	0.0±0.0	0.00±0.00
LSTM-AE	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000	0	0	0.0±0.0	0.00±0.00
GRU-AE	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000	0	0	0.0±0.0	0.00±0.00
Transformer-AE	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000	0	0	0.0±0.0	0.00±0.00

Table II

TEST-SPEAKER RECONSTRUCTION PER INPUT ENCODING (MEAN ± STD OVER FIVE SEEDS).

Model	Encoding	MSE	MAE	R^2
SNN-AE	direct	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000
SNN-AE	poisson	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000
SNN-AE	latency	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000
LSTM-AE	direct	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000
GRU-AE	direct	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000
Transformer-AE	direct	0.0000±0.0000	0.0000±0.0000	0.0000±0.0000

Table III

SNN PRE-PROCESSING MODE, THRESHOLD, AND DECAY SELECTED ON EACH OUTER FOLD'S VALIDATION SPEAKER. A PARENTHESISED FRACTION MARKS FOLDS WHERE THE SEEDS DID NOT UNANIMOUSLY SELECT THE SAME MODE.

Fold	Test spk.	Val spk.	Enc.	Mode	V_{th}	α
0	—	—	—	—	—	—

Table IV

RECORDING-LEVEL PAIRED ANALYSIS ON THE TEST SPEAKERS. $\overline{d_r}$ IS THE MEAN PER-RECORDING MSE DIFFERENCE (MODEL − REFERENCE); A NEGATIVE VALUE FAVOURS THE MODEL. CI IS A 95% BOOTSTRAP INTERVAL OVER RECORDINGS; p_{Holm} IS THE HOLM-CORRECTED WILCOXON SIGNED-RANK p-VALUE.

Model	Reference	n_{rec}	$\overline{d_r}$	95% CI	p_{Holm}
—	—	0	0.0000	[0.0000, 0.0000]	1

and mean-frame linear references, for 1-D temporal signal reconstruction on FSDD. The evaluation is leakage-controlled: nested six-fold leave-one-speaker-out cross-validation with no recording or speaker crossing a split and SNN hyperparameters selected on the inner validation speaker only. Differences are reported at the recording level with bootstrap intervals and Holm-corrected tests, with speaker- and seed-level results as robustness evidence; cost is an analytic per-class operation count, explicitly not an energy measurement. This design directly answers the leakage, missing-baseline, missing-significance, and energy-overclaim objections to the earlier version of this study. The harness, profiles, and analysis scripts are released so the protocol can be reused for other 1-D signal families.

REFERENCES

[1] Anonymous, "Reference suppressed for double-blind review," 2026, code repository citation withheld; to be restored in the camera-ready version.

[2] Z. Jackson *et al.*, "Free spoken digit dataset," GitHub repository, 2018. [Online]. Available: https://github.com/Jakobovski/free-spoken-digit-dataset

[3] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems*, vol. 27, 2014, pp. 3104–3112.

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] W. Maass, "Networks of spiking neurons: The third generation of neural networks," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[6] M. Pfeiffer and T. Pfeil, "Deep learning with spiking neurons: Opportunities and challenges," *Frontiers in Neuroscience*, vol. 12, p. 774, 2018.

[7] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.

[8] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[9] F. Zenke and S. Ganguli, "Superspike: Supervised learning in multilayer spiking neural networks," *Neural Computation*, vol. 30, no. 6, pp. 1514–1541, 2018.

[10] J. K. Eshraghian *et al.*, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[11] xtensor-stack contributors, "xtensor: C++ tensors with broadcasting and lazy computing," GitHub repository, 2026, accessed: 2026-05-07. [Online]. Available: https://github.com/xtensor-stack/xtensor

[12] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.

[13] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "Lstm: A search space odyssey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2222–2232, 2017.
[14] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
[15] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.
[16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
[17] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
[18] F. Zenke and T. P. Vogels, "The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks," *Neural Computation*, vol. 33, no. 4, pp. 899–925, 2021.
[19] G. Bellec, F. Scherr, A. Subramoney, R. Legenstein, and W. Maass, "A solution to the learning dilemma for recurrent networks of spiking neurons," *Nature Communications*, vol. 11, p. 3625, 2020.
[20] S. Thorpe, A. Delorme, and R. Van Rullen, "Spike-based strategies for rapid processing," *Neural Networks*, vol. 14, no. 6–7, pp. 715–725, 2001.
[21] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010.
[22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.